

Proposal Template

TankHell

1. Project Overview

Provide a brief overview of your project, explaining the game's concept, mechanics, and functionality.

I will make a copy of ShellShock Live on Steam, it is a game where two or more tanks shoot at each other in a 2D plane. There are many types of shell or ammo to choose from; splash shell, napalm strike, orbital strike, etc.

2. Project Review

The primary inspiration for this project is an existing game called ShellShock Live, which features two or more tanks shooting at each other in a 2D plane. In the original game, players have access to many types of shells and ammo to choose from, such as splash shells, napalm strikes, and orbital strikes.

My project, TankHell, will build upon this foundation by implementing a progression and unlocking system for the various tank shells, giving players a long-term objective. Additionally, to make the gameplay more dynamic and exciting, the terrain in TankHell can also be destroyed when a shell hits the ground and explodes.

3. Programming Development

3.1 Game Concept

Describe the game in detail, including its mechanics and objectives. Highlight the key features or selling points of the game.

The game will be about tanks shooting each other. The tank will be placed on the different sides of the map and its objective is to shoot each other till someone explodes. While during your turn, you can move your tank a set distance based on how much fuel you have left, then you can choose the type of shell and shoot at the enemy. The shell will move in a projectile motion to the enemy, it will show the projection of the projectile close to your tank. There will be two modes; single player, and two player. When the shell hits the ground and explodes the terrain can also be destroyed to make an interesting and exciting game.

3.2 Object-Oriented Programming Implementation

Tank(): - The tank class is responsible for all player and enemy tank-related data and actions.

- pos: Vector2 - Stores x, y coordinates.
 - angle: float - Barrel aim, typically constrained between 0-180 degrees.
 - power: float - Shot strength, ranging from 0-100.
 - color: str - Used visually to distinguish between player 1 and player 2.
 - player_id: int - A unique ID (e.g., 1 or 2) to easily track stats and turns in the main loop.
 - health: float - Current health points.
 - armor: float - Damage reduction multiplier.
 - oil: float - Fuel remaining, determining how far the tank can travel.
 - is_alive: bool - Status flag for when health reaches zero.
 - shell_list: dict - Inventory of available ammunition.
 - current_shell: str - Tracks which weapon the player currently has equipped.
 - facing_left: bool - Direction the tank sprite is looking.
 - accuracy: float - Stat tracking for player accuracy.
 - time_taken: float - Stat tracking for turn duration.
 - movedistance: float - Stat tracking for total distance moved.
 - rect: Rect - A bounding box for the tank to handle precise terrain collision and getting hit by shells.
-
- draw(surface) - Renders the tank body and the rotating barrel.
 - update() - Checks for gravity to make the tank fall if the ground beneath is destroyed.
 - take_dmg(amount) - Reduces health and armor when hit.
 - shoot(shell, power, angle) - Instantiates and fires a shell object.
 - move(facing_left, oil) - Moves the tank along the terrain based on fuel.

Shell(): - The projectile fired by the tanks.

- type: str - The shell name/category.
 - dmg_peruse: list - Stores the number of uses and the specific damage value.
 - pos: Vector2 - Tracks movement coordinates through the air.
 - velocity: Vector2 - Current speed and direction.
 - gravity: float - Constant downward force.
 - damage: float - Base damage dealt on direct impact.
 - timer: float - Used for delayed explosions or tracking trail effects.
 - explosion_radius: float - Size of the terrain destruction and splash damage area.
 - damage_falloff: float - Calculates how damage reduces further from the blast center.
 - wind_resistance: float - Affects trajectory based on global wind.
 - owner_id: int - (New) Tracks who fired the shell to accurately award damage stats to the correct player.
 - is_active: bool - (New) Flags the shell for deletion once it explodes to prevent memory leaks.
-
- update() - Where the physics happen (e.g., $\text{velocity.y} += \text{gravity}$ and $\text{pos} += \text{velocity}$).
 - check_collision(terrain, tanks) - Checks if the shell hit the ground mask or an enemy rect.

Terrain(): - Handles the destructible environment.

- image: Surface - The visual representation of the terrain.
 - mask: Mask - Used for pixel-perfect collision detection.
 - rect: tuple - The (x, y) coordinates for screen placement.
 - points: list - Stores height data tuples for generating random hilly landscapes.
 - width: int & height: int - (New) Defines map boundaries to despawn shells that fly completely off-screen.
-
- generate_terrain(points) - Creates the surface using Perlin noise or midpoint displacement.
 - destroy(impact_pos, radius) - Makes terrain destructible by cutting a circle out of the image and updating the mask.
 - get_y_at(x) - Checks the mask or surface at an x-coordinate to find the first non-transparent pixel, keeping tanks glued to the ground.
 - draw(screen) - Renders the terrain.

Game(): - The manager class that handles the main game loop and states.

- player_list: list - Stores the active Tank objects.
 - active_shell: Shell - Tracks the currently flying projectile.
 - wind: float - Global wind variable affecting shell trajectories.
 - terrain: Terrain - The current map object.
 - rounds: int - Current round number.
 - matchduration: float - Tracks how long the match takes for statistics.
 - current_turn_index: int - Tracks whose turn it is in the player_list.
 - game_state: str - Tracks if the game is currently in the 'main_menu', 'playing', or 'game_over' screen.
-
- run() - Contains the while loop keeping the game open.
 - update() - Handles background logic, state transitions, and turn switching.
 - draw() - Blitzes the current terrain image, UI, and entities to the screen.
 - quit_game() - Closes the application safely.

Particle():

- color: rgb - Color of the particle.
 - size: int - Size in pixels.
 - lifetime: float - How long the particle exists before vanishing.
 - velocity: Vector2 - Speed and direction.
 - is_dead: bool - (New) Flags the particle for memory cleanup once its lifetime reaches 0.
-
- update() - (New) Moves the particle based on velocity and reduces its lifetime each frame.
 - draw(screen) - Renders the visual particle.

3.3 Algorithms Involved

- **Terrain Generation:** The game will utilize Perlin noise or midpoint displacement algorithms to programmatically generate random, natural-looking hilly landscapes for the tanks to spawn on.
- **Projectile Kinematics:** To calculate the projectile motion of the fired shells, the game will use physics algorithms that apply a constant downward gravity force to the shell's vertical velocity, updating its position frame-by-frame as it travels through the air.
- **AI Behavior Logic:** The enemy tanks will use rule-based logic to make decisions during their turn, evaluating their current fuel limits to decide if they should move, and calculating the necessary barrel angle and shot power to hit the player

4. Statistical Data (Prop Stats)

4.1 Data Features

	Why is it good to have this data? What can it be used for	How will you obtain 100 values of this feature data?	Which variable and which class will you collect this from?	How will you display this feature data (via summarization statistics or via graph)?
accuracy	to see the skill of each player	shoot 100 times	class Tank variable accuracy	pie chart
average time taken per shots	to see the improvement of each player	shoot 100 games	class Tank variable time_taken	line graph
average damage per use	to balance all the shell types	shoot 100 times	class Shell variable dmg_peruse	bar graph
average moving distance	to balance the fuel and tank mobility	play 100 games	class Tank variable movedistance	line graph
match duration	to balance out if the match is taking too long or not	play 100 games	class Game variable matchduration	line graph

4.2 Data Recording Method

Explain how the game will store statistical data. Will it be saved in a database, CSV file, or another format?

The game will store the data in its own specific variable and then it will be made into a dataframe then save it in a csv file.

4.3 Data Analysis Report

Outline how you will analyze the recorded data. What statistical measures will you use? How will the analysis be presented (e.g., graphs, tables, charts)?

	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
accuracy	accuracy	to compare the skill of each player	pie chart	Hit %	Not Hit %
average time taken per shots	time taken per shots	to see the improvement of each player	line graph	Game Number	Time taken per shot
average damage per use	average damage per use	to balance all the shell types	bar graph	Shells	Damage
average moving distance	average moving distance	to balance the fuel and tank mobility	line graph	Game Number	Distance Moved
match duration	match duration	to balance out if the match is taking too long or not	line graph	Game Number	Time taken

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Set up the Pygame project environment and Git repository. Create the main Game() loop, screen rendering, and basic state management.
3	10	Implement the Terrain() class, focusing on generating the random landscape using Perlin noise or midpoint displacement. Create the base Tank() class (spawning on the terrain, basic left/right movement with fuel/oil limits).
4	11	Implement the Shell() class with projectile kinematics (gravity, velocity, and aiming angles). Set up the turn-based logic so players can take turns shooting at each other.
5	12	Implement terrain destruction (updating the mask when a shell collides with the ground). Add the Particle() class for explosion visual effects and basic UI (health bars, fuel gauges, current weapon).
6	13	Set up the stat-tracking variables in your classes (accuracy, time taken, distance moved, match duration). Write the code to automatically export this tracked data into your CSV file at the end

		of each match.
7	14	Submission week (Draft)

6. Document version

Version: 1.0

Date: 12 March 2025

Editor: Suppamok Tosranon

Version: 1.1

Date: 28 March 2025

Editor: Suppamok Tosranon

Date	Name	Description of Revision, Feedback, Comments
14/3	Nattanan	Be more specific with data Here is a framework I have for you. (What, Where, When, Why) What- what data will you collect Where - where will this data store before sent to csv When - when will the data sent to csv (what triggered this thing) Why - Why this is good to have this will make it easier to implement too Also filled the planning too
21/03	Amornrit	Explain briefly about the involved algorithms and each tracked data feature. Fix the inconsistent formatting in the OOP section (the '#' part).

Project Requirement

Requirements (Game project, also year project)

Game Development

- **Game part:** Must contain a **playable game** (designed for general players)
 - **Stats part:** Must include a **statistics collection and useful visualization**
 - Collect data from gameplay
 - Minimum **5 measurable features**
 - Total data collection must be **more than 100 records (each feature)**
 - Data must come from actual gameplay
-

Programming Requirements

- Must use **Object-Oriented Programming (OOP)**
 - Must have **at least 5 classes**
-

Document & Submission Requirements

Students must submit:

- **Proposal**
 - **Final Proposal**
 - **Final Project Report**
 - **UML Diagrams** (Class Diagram + others if applicable)
 - **Video Presentation**
 - **GitHub Repository**
 - **Department Year Project Website Submission**
-

Viewer example project from:

<https://ecourse.cpe.ku.ac.th/tpm/project/prog2-24s>

(It doesn't mean you have to follow these projects)